Assignment 21.3

K . Jashwanth
2303A52300

Batch – 42

Question 1: AI-Assisted Pair Programming

Prompt:

Give code for Expense Tracker.

Code:

```python
import json
import os
class ExpenseTracker:
    def __init__(self, filename='expenses.json'):
        self.filename = filename
        self.expenses = []
        self.load_from_file()
    def add_expense(self, amount, category, description):
        try:
            amount = float(amount)
            if amount <= 0:
                raise ValueError("Amount must be positive")
            self.expenses.append({
                'amount': amount,
                'category': category,
                'description': description
            })
            self.save_to_file()
            print("Expense added successfully!")
        except ValueError as e:
            print(f"Error: {e}")
    def view_expenses(self):
        if not self.expenses:
            print("No expenses recorded yet.")
            return
        print("Expenses:")
        for i, exp in enumerate(self.expenses, 1):
            print(f"{i}. ${exp['amount']:.2f} - {exp['category']} - {exp['description']}")
    def total_expenses(self):
        total = sum(exp['amount'] for exp in self.expenses)
        print(f"Total expenses: ${total:.2f}")
        return total
    def save_to_file(self):
        with open(self.filename, 'w') as f:
            json.dump(self.expenses, f, indent=4)
    def load_from_file(self):
        if os.path.exists(self.filename):
```

```python
class ExpenseTracker:
    def save_to_file(self):
            json.dump(self.expenses, f, indent=4)
    def load_from_file(self):
        if os.path.exists(self.filename):
            try:
                with open(self.filename, 'r') as f:
                    self.expenses = json.load(f)
            except json.JSONDecodeError:
                print("Error loading expenses file. Starting fresh.")
def main():
    tracker = ExpenseTracker()
    while True:
        print("\nExpense Tracker Menu:")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Expenses")
        print("4. Exit")
        choice = input("Choose an option: ").strip()

        if choice == '1':
            amount = input("Enter amount: ").strip()
            category = input("Enter category: ").strip()
            description = input("Enter description: ").strip()
            tracker.add_expense(amount, category, description)
        elif choice == '2':
            tracker.view_expenses()
        elif choice == '3':
            tracker.total_expenses()
        elif choice == '4':
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output:

```
Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Exit
Choose an option: 1
Enter amount: 5000
Enter category: grocerry
Enter description: Household things
Expense added successfully!

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Exit
Choose an option:
```

Prompt:

Develop a Python program for a simple Expense Tracker Application that allows users to record daily expenses along with categories such as food, travel, and shopping. First, create a basic version that takes user input and stores the expense details in a simple format. Then, analyze the limitations of this version, such as lack of input validation and poor structure. Improve the program by adding features like proper input validation, monthly expense summary, and organizing the code using functions. Finally, provide both versions of the program and explain how the improvements make it more efficient and user-friendly.

Code:

```python
import json
import os
from datetime import datetime
from collections import defaultdict

# Improved Expense Tracker
# With input validation, functions, monthly summary, and file storage

def load_expenses(filename='expenses.json'):
    if os.path.exists(filename):
        try:
            with open(filename, 'r') as f:
                return json.load(f)
        except json.JSONDecodeError:
            print("Error loading expenses file. Starting fresh.")
    return []

def save_expenses(expenses, filename='expenses.json'):
    with open(filename, 'w') as f:
        json.dump(expenses, f, indent=4)

def add_expense(expenses):
    while True:
        amount_str = input("Enter amount: ").strip()
        try:
            amount = float(amount_str)
            if amount <= 0:
                print("Amount must be a positive number.")
                continue
            break
        except ValueError:
            print("Invalid amount. Please enter a number.")

    category = input("Enter category (e.g., food, travel): ").strip()
    if not category:
        category = "misc"
```

```python
def add_expense(expenses):

    category = input("Enter category (e.g., food, travel): ").strip()
    if not category:
        category = "misc"

    description = input("Enter description: ").strip()

    date_str = input("Enter date (YYYY-MM-DD) or press Enter for today: ").strip()
    if not date_str:
        date = datetime.now().strftime('%Y-%m-%d')
    else:
        try:
            datetime.strptime(date_str, '%Y-%m-%d')
            date = date_str
        except ValueError:
            print("Invalid date format. Using today's date.")
            date = datetime.now().strftime('%Y-%m-%d')

    expenses.append({
        'amount': amount,
        'category': category,
        'description': description,
        'date': date
    })
    print("Expense added successfully!")

def view_expenses(expenses):
    if not expenses:
        print("No expenses recorded.")
        return
    print("Expenses:")
    for i, exp in enumerate(expenses, 1):
        print(f"{i}. {exp['date']} - ${exp['amount']:.2f} - {exp['category']} - {exp['description']}")

def total_expenses(expenses):
    total = sum(exp['amount'] for exp in expenses)
```

```python
def monthly_summary(expenses):
    for exp in expenses:
        month = exp['date'][:7]  # YYYY-MM
        monthly[month] += exp['amount']
    print("Monthly Summary:")
    for month, total in sorted(monthly.items()):
        print(f"{month}: ${total:.2f}")

def main():
    expenses = load_expenses()
    while True:
        print("\nExpense Tracker Menu:")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Expenses")
        print("4. Monthly Summary")
        print("5. Exit")
        choice = input("Choose an option: ").strip()

        if choice == '1':
            add_expense(expenses)
            save_expenses(expenses)
        elif choice == '2':
            view_expenses(expenses)
        elif choice == '3':
            total_expenses(expenses)
        elif choice == '4':
            monthly_summary(expenses)
        elif choice == '5':
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output:

```
Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Monthly Summary
5. Exit
Choose an option: 1
Enter amount: 2000
Enter category (e.g., food, travel): food
Enter description: junk food
Enter date (YYYY-MM-DD) or press Enter for today:
Expense added successfully!

Expense Tracker Menu:
1. Add Expense
2. View Expenses
3. Total Expenses
4. Monthly Summary
5. Exit
Choose an option:
```

JUSTIFICATION:

An expense tracker helps in understanding how real-world applications are developed. A basic version may lose data and accept invalid inputs, which reduces reliability. By adding input validation, data storage, and structured code, the program becomes more efficient and user-friendly. Generating summaries also helps in analyzing spending patterns effectively. Overall, this task builds practical skills like error handling, data management, and clean coding used in professional software development.

Question 2: Version Control Integration

Prompt:

Generate a code for python based file upload script. Identify vulnerabilities such as, No file type validation, Unrestricted file upload.

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> PS C:\Users\pambi\OneDrive\Documen
ts\Desktop\AI-Assited-Coding> cd C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\stude
nt_grade_calculator
>>
>> PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\student_grade_calculator> git in
it
>>
>> PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\student_grade_calculator> git ad
d .
>>
>> PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\student_grade_calculator> git co
mmit -m "Initial commit"
```

Output:

```
Enter the number of subjects: 5
Enter total marks per subject (default 100): 100
Enter marks obtained for subject 1: 90
Enter marks obtained for subject 2: 80
Enter marks obtained for subject 3: 70
Enter marks obtained for subject 4: 60
Enter marks obtained for subject 5: 50
Average Grade: 70.00
Percentage: 70.00%
Grade Classification: C
```

JUSTIFICATION:

The student grade calculator project demonstrates practical Python programming concepts such as input handling, calculations, and the use of functions for better structure. It also integrates version control using Git to highlight professional practices like committing, branching, and merging. This ensures better code management, collaboration, and reduces errors during development. The project is designed simply to focus on core functionality while remaining flexible for future improvements.

Question 3: AI for Commit Message Generation

Prompt:

Generate a Python module for password validation that initially performs a basic check for user input. Then, enhance the module by adding rules such as minimum password length, inclusion of uppercase and lowercase letters, digits, and special characters. Finally, use AI to create clear and meaningful Git commit messages that describe each improvement made to the validation process.

Code:

```
# Password validation module import re
 def validate_password(password):    #
Check minimum length    if len(password)
< 8:
      return False, "Password must be at least 8 characters long."
   # Check for special characters    if not
re.search(r'[!@#$%^&*()-+]', password):
      return False, "Password must contain at least one special character."
   # Check for uppercase letters    if not re.search(r'[A-
Z]', password):
      return False, "Password must contain at least one uppercase letter."
   # Check for lowercase letters    if not re.search(r'[a-
z]', password):
      return False, "Password must contain at least one lowercase letter."

   # Check for digits    if not re.search(r'\d',
password):
      return False, "Password must contain at least one digit."
       return True, "Password is valid."
# Example usage password = input("Enter a password: ")
is_valid, message = validate_password(password)
print(message)
# Git commit message: "Added password validation rules for minimum length, special characters, uppercase
letters, lowercase letters, and digits."
```

AI Committed Message:

```
feat: Enhance password validation with length and special character rules

- Add minimum length requirement of 8 characters
- Require at least one special character using regex
- Import re module for pattern matching
- Improve security against weak passwords
```

Human Version:Added strong password validation rules

JUSTIFICATION:
The password validator module is designed to improve application security by enforcing strong password rules. It begins with basic validation to ensure the password is not empty and then includes checks for minimum length and special characters. These enhancements help prevent weak passwords and protect against common attacks such as brute force. The modular structure makes it easy to integrate into login or registration systems, improving overall reliability and security.

Question 4: Pair Programming Simulation

Prompt:

Generate an HTML feedback form with JavaScript output.

Code:

```python
""" Simple Quiz Program: A quiz with score calculation and input validation
questions = [
    {
        "question": "What is the capital of France?",
        "options": ["A) Paris", "B) London", "C) Berlin", "D) Madrid"],
        "answer": "A"
    },
    {
        "question": "What is 2 + 2?",
        "options": ["A) 3", "B) 4", "C) 5", "D) 6"],
        "answer": "B"
    }
]
def get_valid_answer():
    """ Get a valid answer from user (A, B, C, or D). """
    while True:
        answer = input("Your answer (A/B/C/D): ").strip().upper()
        if answer in ['A', 'B', 'C', 'D']:
            return answer
        else:
            print("Invalid input. Please enter A, B, C, or D.")
def run_quiz():
    score = 0
    total_questions = len(questions)
    for q in questions:
        print(q["question"])
        for option in q["options"]:
            print(option)
        answer = get_valid_answer()
        if answer == q["answer"]:
            print("Correct!")
            score += 1
        else:
            print("Incorrect.")
    percentage = (score / total_questions) * 100
    print(f"Your score: {score}/{total_questions} ({percentage:.1f}%)")
if __name__ == "__main__":
```

Output:

```
3137/python.exe d:/veekshitha/AI_Coding/quiz_program.py
What is the capital of France?
A) Paris
B) London
C) Berlin
D) Madrid
Your answer (A/B/C/D): A
Correct!
What is 2 + 2?
A) 3
B) 4
C) 5
D) 6
Your answer (A/B/C/D): B
Correct!
Your score: 2/2 (100.0%)
```

JUSTIFICATION: This task focuses on generating clear results through score calculation, making performance easy to monitor and assess. It improves reliability by incorporating input validation to handle incorrect entries and reduce errors. Additionally, it emphasizes practical development skills by using Git workflows such as branching, committing, and merging, which support effective teamwork and code management.